

BFF Solution Design: Multi-Services Insurance Platform

About this document

This is my solution design for a Backend-for-Frontend (BFF) that sits in front of the existing Quotation and Payment microservices. It supports two portals: a Customer Portal where end users buy a policy directly, and an Agent Portal where agents do the same on behalf of a customer.

The design uses a single .NET 8 BFF that aggregates both microservices and shapes responses per portal. The sections below cover the architecture, the technology choices, deployment, data handling and PII, observability, and security.

Table of Contents

1. [Context and Assumptions](#)
 2. [Architecture](#)
 3. [Technologies](#)
 4. [Deployment](#)
 5. [Data Handling and PII](#)
 6. [Observability](#)
 7. [Security, Authentication and Authorization](#)
 8. [Summary](#)
-

1. Context and Assumptions

Business context

The Customer Portal lets end users get a quote, customise their coverage, pay, and receive a policy document. The Agent Portal supports the same flow plus a few extras that only agents need: customer search, multi-quote handling, and discount overrides.

Both portals talk to the same Quotation and Payment microservices, but each one needs different request shapes, validation, response shapes and authorization rules. That's the gap the BFF fills.

Rough scale to design for

Around 50,000 quotes a day with peak traffic of about 50 RPS at the BFF. The targets are 99.95% availability and a p95 quote latency under 400 ms. One Azure region to start; multi-region can be revisited if availability requirements demand it.

Existing microservices

The endpoints below are my own assumption, since the brief says to make one. The BFF talks to them through Refit-typed HTTP clients, so if the actual contracts differ the change is contained in one place.

Quotation Service

Method	Endpoint	Purpose
GET	<code>/products</code>	Product catalog and rating factors
POST	<code>/quotes</code>	Create a quote
GET	<code>/quotes/{id}</code>	Fetch a quote
PUT	<code>/quotes/{id}</code>	Update coverage / re-price
POST	<code>/quotes/{id}/bind</code>	Convert a quote into a policy

Payment Service

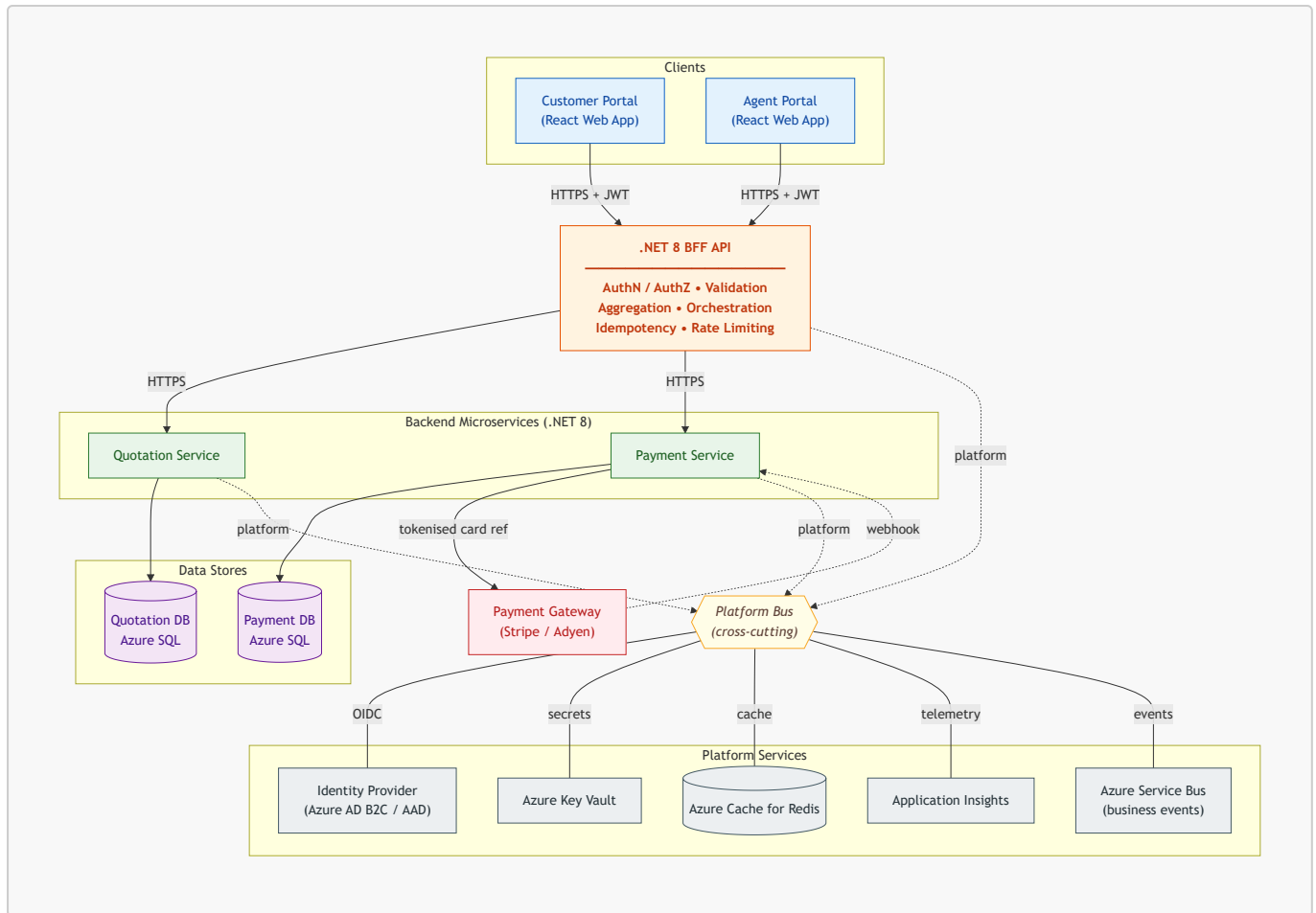
Method	Endpoint	Purpose
POST	<code>/payment-intents</code>	Create a payment intent for a quote
POST	<code>/payment-intents/{id}/confirm</code>	Confirm payment using a gateway token
GET	<code>/payments/{id}</code>	Payment status
POST	<code>/refunds</code>	Issue a refund
POST	<code>/webhooks/gateway</code>	Asynchronous callback from the payment gateway

Other assumptions

- One .NET 8 BFF for both portals, with portal-specific endpoint groups (`/customer/...`, `/agent/...`) and separate authorization policies. The code stays modular enough that we could split it into two BFFs later if the release cadences ever pull apart.
- The browser sends card details straight to the payment gateway (Stripe or Adyen). Our services only ever see the gateway-issued token, never the card number or CVV.
- An OIDC identity provider is in place: Azure AD B2C for customers, corporate Azure AD for agents.
- Azure as the cloud target. The IdP, App Service, Key Vault and Application Insights all live in the same ecosystem, which keeps things simple.

2. Architecture

2.1 Architecture diagram



A couple of things to call out about the diagram:

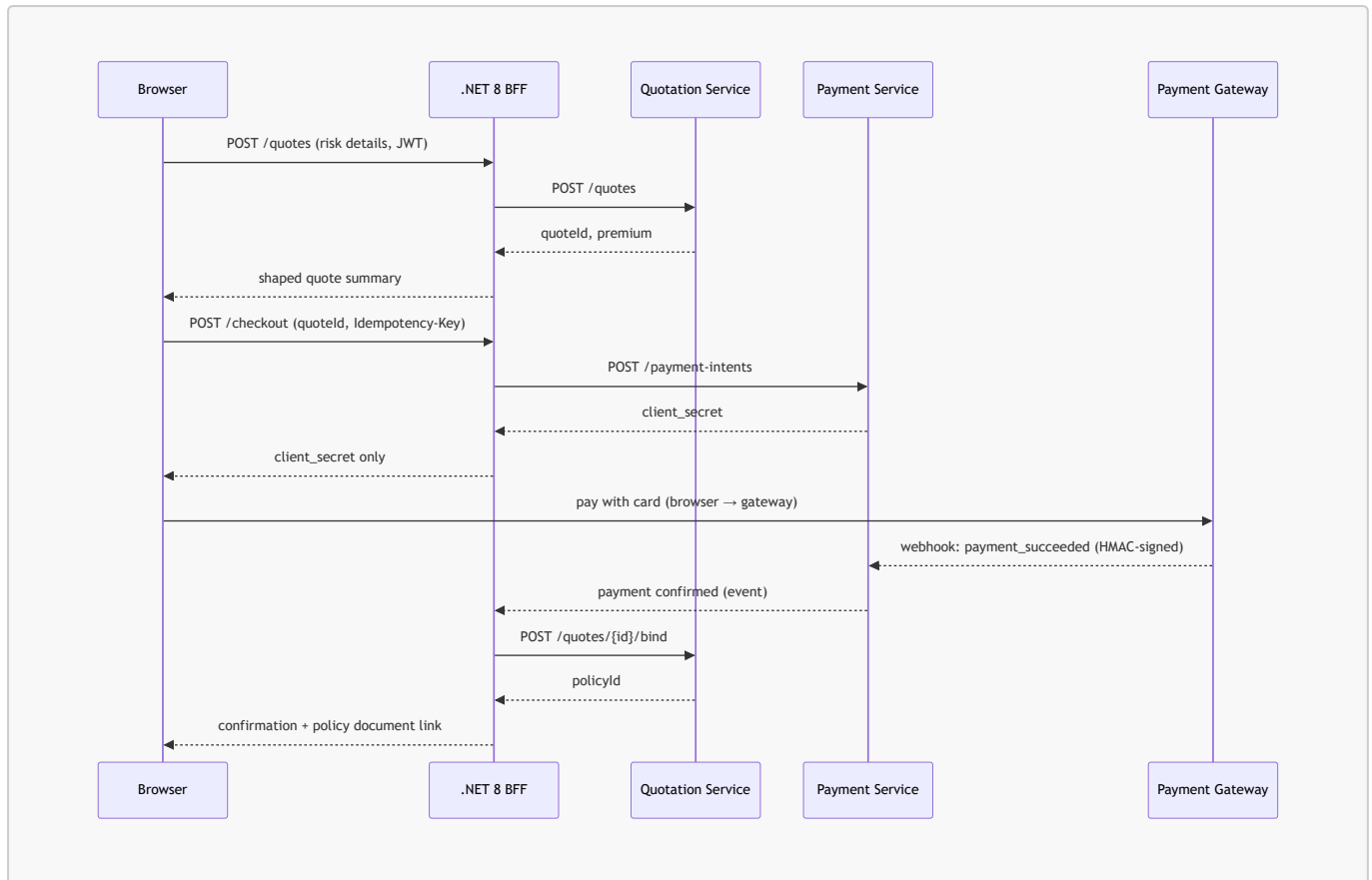
- Solid arrows are synchronous request flows; dashed arrows are asynchronous or cross-cutting.
- The "Platform Bus" node is just a drawing trick to keep things readable. There's no actual bus; each component talks to Key Vault, the IdP, Redis, App Insights and Service Bus directly.
- The Payment Service is the only piece that talks to the external payment gateway.

2.2 What the BFF actually does

The BFF is responsible for a small, well-defined set of things:

- Validates the JWT from the IdP and enforces both role-based and attribute-based authorization.
- Validates incoming requests with FluentValidation, so bad input is rejected at the trust boundary.
- Aggregates Quotation and Payment data into a single response shaped for whichever portal is calling.
- Orchestrates multi-step flows like quote → pay → bind.
- Honours an **Idempotency-Key** header on POSTs by caching the key/response pair in Redis.
- Rate-limits per IP and per user.
- Emits structured logs, metrics and distributed traces with a correlation ID propagated to the downstream services.

2.3 Walkthrough: buying a policy



The card details go straight from the browser to the gateway. Our services only ever see the gateway's token, so card data never lands on a server we run.

3. Technologies

The BFF is .NET 8 (LTS) so it shares the language and toolchain already used by the microservices.

Layer	Choice	Why
Runtime / language	.NET 8 LTS, C# 12	LTS, fast, familiar
BFF API	ASP.NET Core 8 (Minimal APIs)	Less boilerplate than controllers, OpenAPI built-in
Service-to-service calls	<code>IHttpClientFactory</code> + Refit	Strongly typed clients for Quotation and Payment APIs
Resilience	Polly via <code>Microsoft.Extensions.Resilience</code>	Standard retry, timeout, circuit breaker pipelines
Validation	FluentValidation	Readable rules, easy to unit test
Cache	Azure Cache for Redis	Idempotency keys and lightweight read-through cache

Layer	Choice	Why
Authentication	OAuth2 / OIDC via Microsoft.Identity.Web	Standard library for JWT validation in ASP.NET Core
Authorization	ASP.NET Core Authorization Policies + IAuthorizationHandler	Roles plus fine-grained checks, all unit-testable
Secrets	Azure Key Vault + Managed Identity	No secrets in source or config
Logging / metrics / traces	Serilog (JSON) + OpenTelemetry .NET SDK → Application Insights	One backend, all three signals correlated by traceId
Containers	Docker on mcr.microsoft.com/dotnet/aspnet:8.0	Microsoft's official base image
Hosting	Azure App Service with deployment slots	Blue/green and autoscale without managing a cluster
CI / CD	Azure DevOps Pipelines (multi-stage YAML)	One tool, one audit trail
IaC	Bicep	Azure-native, simpler than Terraform if we're sticking to one cloud
Edge	Azure Front Door (CDN + WAF)	TLS termination, OWASP rules, DDoS protection
API Gateway	Azure API Management	OAuth2 enforcement and rate limiting in one place

Project layout

```

src/
  Bff.Api/           ASP.NET Core 8 BFF (Customer + Agent endpoint
groups)
  Bff.Application/   MediatR handlers, validators, mappers
  Bff.Infrastructure/ Refit clients, Redis, telemetry
  Bff.Domain/        Cross-cutting models and contracts
tests/
  *.UnitTests / *.IntegrationTests
deploy/
  bicep/  azure-pipelines/

```

4. Deployment

4.1 Topology

The BFF runs on Azure App Service with separate staging and production slots, so a deploy is a slot swap rather than downtime. Redis, Key Vault and Application Insights are PaaS, so there's nothing to patch.

Outbound calls to the Quotation and Payment microservices go over the internal VNet via private endpoints; the BFF is the only piece exposed to the public internet (through API Management and Front Door).

4.2 Pipeline

A single multi-stage YAML pipeline in Azure DevOps owns build and deploy. The Environments feature gives me approval gates and an audit trail per environment.

```
PR opened
└─ CI: dotnet build → unit tests → SonarCloud
    → docker build → Trivy image scan → push to ACR

Merge to main
└─ Deploy to dev    (auto)
└─ Deploy to uat    (manual approval)
└─ Deploy to prod   (manual approval + change reference)

Each deploy stage:
→ deploy artifact to "staging" slot
→ run smoke tests against the slot
→ swap "staging" ↔ "production" (instant cutover)
→ rollback by swapping slots back if errors spike
```

A few details worth mentioning: variable groups are linked to Key Vault so secrets are pulled at deploy time rather than stored in pipeline definitions, and the service connections to ACR and the Azure subscription use Workload Identity Federation so there are no long-lived credentials sitting around.

4.3 Resilience

Each named `HttpClient` calling Quotation or Payment goes through a Polly pipeline with retry (with jitter), timeout and circuit breaker. Every POST to the Payment Service carries an `Idempotency-Key`; the BFF caches the `key → response` pair in Redis for 24 hours so a network-induced retry doesn't double-charge anyone.

The apps expose `/health/live` and `/health/ready` so App Service can pull a sick instance out of rotation. Shutdown is wired through `IHostApplicationLifetime` to drain in-flight requests during a slot swap, and Serilog gets `Log.CloseAndFlush()` before the process exits.

5. Data Handling and PII

The BFF doesn't own any business data; Quotation and Payment do. What the BFF is responsible for is moving data safely between the browser and the microservices, and not leaking PII through its own logs or responses.

5.1 In-transit

Everything is TLS 1.2+ end to end: browser to BFF (via Front Door / API Management) and BFF to the microservices over the internal VNet. Sensitive data only ever travels in the request body, never in URLs or query strings, and query strings are scrubbed from logs.

5.2 PII handling at the BFF

- A Serilog enricher masks any property tagged with a custom `[Pii]` attribute before the log line is written. An NRIC ends up as `S****1234A` rather than the real value.
- PII never travels in URLs or query strings; the BFF only accepts it in request bodies.
- The BFF returns only the fields the calling portal needs, so we don't leak data the UI didn't ask for.
- Idempotency-key payloads cached in Redis store the *response shape* with PII already masked, never the raw body.

5.3 Card data

The browser uses the gateway's hosted SDK (Stripe Elements or Adyen Drop-in) to collect card details and POSTs them straight to the gateway. The card number and CVV never touch the BFF. The BFF only ever handles the gateway-issued token reference when calling the Payment Service.

6. Observability

6.1 Instrumentation

Logs, metrics and traces all go to Application Insights. The OpenTelemetry SDK auto-instruments the things you'd expect (ASP.NET Core, HttpClient, EF Core, Redis, Service Bus), and W3C Trace Context propagates the same `traceId` across the browser → BFF → microservice → database hops.

Signal	How	What it captures
Traces	OpenTelemetry .NET SDK auto-instrumentation → Application Insights	Browser to BFF to microservice to DB, all linked by one <code>traceId</code>
Metrics	<code>System.Diagnostics.Metrics.Meter</code> → Application Insights	RPS, error rate, latency percentiles per endpoint, GC pauses, ThreadPool starvation, dependency latency
Logs	Serilog (JSON) + <code>Serilog.Sinks.OpenTelemetry</code> → Application Insights	Every log enriched with <code>TraceId</code> , <code>SpanId</code> , <code>service.name</code> , <code>deployment.slot</code> . PII redacted before write.
Browser	Application Insights JS SDK	Web Vitals (LCP, INP, CLS), JS errors, correlated to backend traces
Health checks	<code>Microsoft.Extensions.Diagnostics.HealthChecks</code>	<code>/health/live</code> , <code>/health/ready</code> for App Service
Synthetic	Azure Monitor availability tests	"Buy a policy" run end-to-end every 5 minutes

6.2 Performance targets

Journey	Metric	Target
Get a quote	p95 latency	≤ 400 ms
Pay and bind	success ratio	$\geq 99.9\%$
Agent search	p95 latency	≤ 800 ms
BFF availability	uptime	99.95%

6.3 Alerting

Alerts fire when one of those targets is breached for a sustained window. The preference is to alert on user-visible symptoms (errors, latency, payment success rate) rather than infrastructure causes like CPU or memory, since those generate noise that doesn't map to user pain. Every alert links to a runbook.

6.4 Dashboards

Five dashboards cover the day-to-day:

1. **Business:** quotes per hour, conversion rate, payment success rate.
2. **Per-service:** Rate, Errors, Duration plus a dependency latency heatmap.
3. **Payment funnel:** intent → confirm → webhook → bind, showing where customers drop off.
4. **Infrastructure:** App Service instances, DB DTU, Redis memory, Service Bus queue depth.
5. **Security:** auth failures, WAF blocks, anomalous geographies.

6.5 Tracing discipline

Spans get tagged with business attributes like `quote.id`, `policy.id`, `payment.intent.id`, and `portal=customer|agent`. When a customer reports an issue, you can search by their quote ID and land directly on the trace.

7. Security, Authentication and Authorization

7.1 The basics

Baseline controls applied from day one:

- HTTPS / TLS 1.2+ everywhere, with HSTS at the edge.
- JWTs validated at API Management and again at the BFF. Two checks rather than relying on a single layer; the cost is negligible.
- Managed Identity for every Azure resource access (SQL, Key Vault, Service Bus). No connection strings or shared keys in config.
- Secrets only in Key Vault. Azure DevOps variable groups link through to Key Vault rather than holding values themselves.
- Input validation at the BFF using FluentValidation, so malformed input doesn't reach the downstream services.
- Trivy image scanning and SonarCloud static analysis both gate every deploy.
- OWASP API Top-10 controls applied to every endpoint.

7.2 Identity

Persona	IdP	Login flow	.NET library
Customer	Azure AD B2C (social and email; MFA optional)	OIDC Auth Code + PKCE	<code>Microsoft.Identity.Web</code>
Agent	Corporate Azure AD (mandatory MFA + Conditional Access)	OIDC Auth Code + PKCE	<code>Microsoft.Identity.Web</code>
Service-to-service	Azure Managed Identity	Token from IMDS	<code>Azure.Identity</code> (<code>DefaultAzureCredential</code>)

7.3 Tokens

Access tokens are short-lived JWTs (15 minutes), audience `bff-api`, RS256-signed, validated at API Management and the BFF. Refresh tokens live in a `HttpOnly, Secure, SameSite=Strict` cookie, rotate on each use, and a family-revocation rule kicks in if a token gets reused. Tokens are never put in `localStorage`; the BFF acts as the OAuth client.

7.4 Authorization

Coarse-grained roles (`customer`, `agent`, `agent-supervisor`, `back-office`) come from the JWT and are enforced via ASP.NET Core Authorization Policies and `[Authorize(Policy = "...")]` attributes. That's enough for "can this user even hit this endpoint", but not enough for "can this specific agent see this specific customer".

For the second question, `IAuthorizationHandler` + `IAuthorizationRequirement` handle the check. The rule for a customer-acting-on-behalf-of action is something like `customer.assignedAgentId == agent.id` OR an active delegation exists. That code is unit-testable and lives next to the rest of the application logic.

A few high-risk actions (large refunds, payments above a threshold, agent discount overrides) require fresh MFA. The handler looks at the `auth_time` claim and, if it's stale, returns `403` with `WWW-Authenticate: Bearer error="insufficient_user_authentication"` to trigger a step-up.

One small rule that prevents a lot of trouble: every query gets the customer identifier from the `ClaimsPrincipal`, never from the request body. That single rule blocks the most common API vulnerability (an attacker tampering with an ID in the request).

7.5 API hardening

Rate limiting is enforced at API Management, per IP, per user, per route, with stricter limits on `/quotes` and `/payment-intents`. CORS is an explicit allow-list of portal origins, no wildcards. The edge sets the usual security headers (HSTS, CSP, X-Frame-Options, Referrer-Policy). For cookie-based sessions, `SameSite=Strict` plus a double-submit token guards against CSRF. And every webhook from the payment gateway is HMAC-verified with a replay window.

7.6 Auditing

Every privileged BFF action (an agent acting on a customer, initiating a refund, a discount override) is written to an append-only audit log. Authentication anomalies (impossible travel, brute force, token reuse) surface

through Application Insights.

8. Summary

A single .NET 8 BFF sits behind Azure Front Door (CDN + WAF) and Azure API Management, in front of the existing Quotation and Payment microservices. The browser tokenises card details directly with the gateway, so card data never reaches our platform.

Topic	Choice	Why
Architecture	One .NET 8 BFF with portal-aware endpoint groups	Smaller deployment surface; still tailored per portal
Stack	ASP.NET Core 8 Minimal APIs, Refit, Polly, Serilog, OpenTelemetry	Familiar .NET ecosystem
Hosting	Azure App Service with deployment slots	Blue/green and autoscale without operating a cluster
CI / CD	Azure DevOps Pipelines with slot swap	One tool, one approval flow
Reliability	Polly retry/timeout/circuit-breaker + idempotency keys in Redis	Safe retries against Payment without double-charging
Security	OIDC, Managed Identity, Key Vault, RBAC + ABAC, mandatory MFA for agents	Defence in depth, least privilege
Payments	Browser-to-gateway tokenisation	Card data never reaches the BFF
Observability	OpenTelemetry + Serilog → Application Insights, alerts on user-visible symptoms	One backend; alerts that map to actual user pain

A few decisions worth calling out

- **Single BFF over two per-portal BFFs.** Portal-specific behaviour expresses cleanly through endpoint groups, validation profiles and authorization policies. The codebase is structured so we can split it later if release cadences pull apart.
- **App Service over AKS.** At this service count, a Kubernetes cluster is more operational overhead than benefit. AKS becomes the right call once we outgrow App Service slots and built-in autoscale.
- **Browser-side card tokenisation.** Probably the most consequential decision in the design. Card data never touches our infrastructure.
- **Outbox + idempotency over distributed transactions.** The BFF stamps every payment-mutating call with an **Idempotency-Key** and caches the response in Redis, so retries are safe end to end without coordinating a transaction across services.
- **Single Azure region to start.** Multi-region is significant additional work and should be deferred until availability requirements actually demand it.